

Securing and Scaling a Production-Style Web Infrastructure

Web Infrastructure — Group 07

This project implements a small production-style web infrastructure using **NGINX**, **two Python backend servers**, **HTTPS/TLS** and **UFW firewall rules** on Parrot Security OS.

The main purpose of the project was to understand how a web request moves from a client through a reverse proxy and load balancer to backend services, while also applying basic security and troubleshooting practices.

Project Overview

The infrastructure contains:

- **NGINX** as a reverse proxy and load balancer
- **Backend 1** running on `127.0.0.1:3001`
- **Backend 2** running on `127.0.0.1:3002`
- **HTTPS** using a self-signed TLS certificate
- **UFW** firewall for controlling network access
- **Systemd** services for managing the backend applications
- **Parrot Security OS** as the host environment

The backend services are not directly exposed to external clients. Requests go through NGINX first.

Request Flow

```
Client
  |
  | HTTP :80
  | 301 Redirect
  v
HTTPS :443
  |
  v
UFW Firewall
  |
  v
NGINX
Reverse Proxy + Load Balancer
  |
```



Backend Services

Two simple Python HTTP services were created.

Backend 1

127.0.0.1:3001

Response:

Web Infrastructure - Group_07 - Server 1

Backend 2

127.0.0.1:3002

Response:

Web Infrastructure - Group_07 - Server 2

Both services are managed using systemd and configured with:

Restart=on-failure

They also run using the unprivileged `www-data` user.

NGINX Reverse Proxy and Load Balancing

NGINX is used as the public entry point for the application.

The two backend servers are placed in an upstream pool:

```
upstream backend_pool {  
    server 127.0.0.1:3001;  
    server 127.0.0.1:3002;  
}
```

Requests are then passed to the backend pool:

```
location / {  
    proxy_pass http://backend_pool;  
    proxy_set_header Host $host;  
    proxy_set_header X-Real-IP $remote_addr;  
}
```

Ten repeated requests were used to test the load balancing. The responses alternated between Server 1 and Server 2.

Backend 1 was also stopped during testing. NGINX continued serving requests through Backend 2.

HTTPS / TLS

A self-signed certificate was created using OpenSSL.

The certificate uses:

- RSA 2048-bit key
- 365-day validity
- localhost
- 127.0.0.1

The private key was protected using:

```
chmod 600
```

HTTP traffic on port 80 is redirected to HTTPS on port 443:

```
server {  
    listen 80;  
    return 301 https://$host$request_uri;  
}
```

The HTTPS server uses:

```
listen 443 ssl;  
ssl_protocols TLSv1.2 TLSv1.3;
```

The certificate was checked using OpenSSL commands to confirm its details and verify the certificate being served by NGINX.

Firewall Configuration

UFW was configured to allow the ports required by the infrastructure.

Allowed

```
22/tcp    SSH  
80/tcp    HTTP  
443/tcp   HTTPS
```

Denied

```
3001/tcp  Backend 1  
3002/tcp  Backend 2
```

The default firewall policy was configured as:

```
Incoming: DENY  
Outgoing: ALLOW
```

The backend services also listen only on `127.0.0.1`, providing an additional layer of protection.

Troubleshooting

Two troubleshooting cases were completed.

Case 1 — NGINX Configuration Failure

NGINX failed to reload because of a problem with the main:

```
/etc/nginx/nginx.conf
```

The issue was investigated using:

```
nginx -t  
systemctl status nginx  
journalctl
```

The configuration was inspected and corrected.

The fix was verified using:

```
nginx -t
```

which returned:

```
syntax is ok  
test is successful
```

NGINX was then reloaded successfully.

Case 2 — Backend 2 Marked as Backup

A deliberate fault was introduced by changing Backend 2 to:

```
server 127.0.0.1:3002 backup;
```

After reloading NGINX, repeated requests returned only Server 1.

Backend 2 was tested directly and was still working.

Backend 1 was then stopped, after which Backend 2 started responding. This confirmed that Backend 2 was functioning as a backup server.

The `backup` directive was removed and the configuration was tested again.

Normal load balancing was restored, with requests once again reaching both Server 1 and Server 2.

Testing Performed

The following tests were carried out during the project:

- Backend service status checks
- Listening port checks using `ss -tlnp`
- Direct curl tests to both backends
- NGINX configuration testing using `nginx -t`
- HTTP to HTTPS redirect testing
- HTTPS request testing
- Ten-request load-balancing tests
- Backend failure testing
- TLS certificate inspection
- UFW rule verification
- Direct access tests to backend ports
- Troubleshooting and recovery tests

Technologies Used

Technology	Purpose
Parrot Security OS	Host operating system
NGINX	Reverse proxy and load balancer
Python	Backend HTTP services
Systemd	Backend service management
OpenSSL	TLS certificate generation and inspection
UFW	Firewall configuration
Curl	HTTP/HTTPS testing
SSH	System administration
Linux	Infrastructure environment

Project Structure

The submission contains the configuration files, backend code, evidence and documentation used during the project.

Group_07/
|

```
├── backend1/
│   └── server.py
├── backend2/
│   └── server.py
├── nginx/
│   └── configuration files
├── tls/
│   └── nginx-selfsigned.crt
├── evidence/
│   └── screenshots and test outputs
├── environment_versions.txt
└── README.md
```

The private TLS key is not included in the submission.

Video Demonstration

The recorded demonstration of the project is available here:

Google Drive:

<https://drive.google.com/file/d/1VF7UpZ5UFVg3mfAHzFNBtSp4oV1RMC4h/view?usp=drivesdk>

Group

Group 07

Submitted by: Jacintha Ngugi

Module: Web Infrastructure

Date: 21 September 2026

AI Transparency

AI assistance was used during the preparation of the project documentation to help with report structure, understanding some NGINX, UFW and OpenSSL output, and organizing the information.

The configuration, commands, testing and troubleshooting were carried out by the group on the Parrot OS environment.